PYTHON PROGRAMMING

Activity-Based Learning (ABL) Assignment

Dr. RVR NRI Institute of Technology

10 Case Studies — Source Code, Algorithms, Flowcharts, Test Cases and Output Screenshots

Assignment Contents

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Employee Payroll and Deduction Analysis
4. Retail Billing with Multiple Discount Rules
5. Attendance Recovery Computation
6. ATM Transaction Sequence Simulation
7. Warehouse Inventory Reconciliation
8. Student Marks Matrix Analysis
9. Password Policy Validation Engine
10. Campus Event Financial Planning

Case Study 01: University Scholarship Allocation

1. Problem Statement

Design a rule-based system that determines scholarship eligibility and estimates the award amount from a student's CGPA, annual family income, and category. The rules are illustrative academic rules, not official university policy.

2. Objectives

• Validate student inputs
• Apply multiple eligibility conditions
• Calculate an award using a percentage cap
• Explain why an applicant qualifies or does not qualify

3. Assumptions

• CGPA is between 0 and 10 inclusive.
• Annual family income cannot be negative.
• Eligibility requires CGPA >= 8.0 and annual family income <= ■300,000.
• Eligible students receive 50% of the annual fee, capped at ■40,000.

4. Algorithm

1. Read CGPA, annual family income, and annual fee.
2. Reject invalid values.
3. Check the CGPA and income conditions.
4. If eligible, calculate min(50% of fee, ■40,000); otherwise award zero.
5. Return eligibility, reason, and amount.

5. Flowchart (logical sequence)

Start → Validate CGPA, income and fee → CGPA >= 8.0? → Income <= ■300,000? → Award = min(50% fee, ■40,000) → Return eligibility and award → End

6. Pseudocode

```
FUNCTION allocate(cgpa, income, fee)\n  VALIDATE 0 <= cgpa <= 10 AND income >= 0 AND fee >= 0\n  IF cgpa < 8.0: RETURN not eligible, reason\n  IF
```

7. Python Source Code

Full executable source: programs/scholarship_allocation.py

```
def allocate_scholarship(cgpa, annual_income, annual_fee):
    """Return eligibility, reason, and scholarship amount in rupees."""
    if not 0 <= cgpa <= 10:
        raise ValueError("CGPA must be between 0 and 10.")
    if annual_income < 0 or annual_fee < 0:
        raise ValueError("Income and fee cannot be negative.")
    if cgpa < 8.0:
        return {"eligible": False, "reason": "CGPA below 8.0", "award": 0.0}
    if annual_income > 300000:
        return {"eligible": False, "reason": "Income exceeds ■300,000", "award": 0.0}
    award = min(annual_fee * 0.50, 40000.0)
    return {"eligible": True, "reason": "All criteria satisfied", "award": round(award, 2)}

def main():
    samples = [
        ("Eligible, fee ■70,000", 8.6, 250000, 70000),
        ("Low CGPA", 7.5, 200000, 70000),
        ("Income too high", 9.2, 350000, 70000),
        ("Award cap", 9.0, 100000, 100000),
    ]
    print("UNIVERSITY SCHOLARSHIP ALLOCATION")
    for label, cgpa, income, fee in samples:
        result = allocate_scholarship(cgpa, income, fee)
        print(f"{label}: eligible={result['eligible']}, award=■{result['award']:.2f} ({result['reason']})")

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Eligible student	CGPA=8.6, income=250000, fee=70000	Eligible; award ■35,000.00
2. CGPA below threshold	CGPA=7.5, income=200000, fee=70000	Not eligible; award ■0.00
3. Income above limit	CGPA=9.2, income=350000, fee=70000	Not eligible; award ■0.00
4. Maximum award cap	CGPA=9.0, income=100000, fee=100000	Eligible; award ■40,000.00

9. Sample Execution Output

```
UNIVERSITY SCHOLARSHIP ALLOCATION
Eligible, fee ■70,000: eligible=True, award=■35000.00 (All criteria satisfied)
Low CGPA: eligible=False, award=■0.00 (CGPA below 8.0)
Income too high: eligible=False, award=■0.00 (Income exceeds ■300,000)
Award cap: eligible=True, award=■40000.00 (All criteria satisfied)
```

Python output — Case Study 01

```
> python scholarship_allocation.py
UNIVERSITY SCHOLARSHIP ALLOCATION
Eligible, fee ₹70,000: eligible=True, award=₹35000.00 (All criteria satisfied)
Low CGPA: eligible=False, award=₹0.00 (CGPA below 8.0)
Income too high: eligible=False, award=₹0.00 (Income exceeds ₹300,000)
Award cap: eligible=True, award=₹40000.00 (All criteria satisfied)
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 02: Progressive Electricity Billing

1. Problem Statement

Calculate a household electricity bill using progressive consumption slabs, where units in each slab are charged at that slab's rate.

2. Objectives

• Implement tiered billing correctly
• Use loops to process slabs
• Validate units and fixed charges
• Display energy charge and final bill

3. Assumptions

• 0–100 units: ■1.50/unit; 101–200: ■2.50/unit; above 200: ■4.00/unit.
• A fixed charge of ■75 is added to every bill.
• Units and fixed charge must be non-negative.

4. Algorithm

1. Validate units and fixed charge.
2. Process each slab in order.
3. Charge only the units that fall within that slab.
4. Add the fixed charge.
5. Return the itemized total.

5. Flowchart (logical sequence)

Start → Read consumed units → Units non-negative? → Calculate charge slab by slab → Add fixed charge ■75 → Display itemized bill → End

6. Pseudocode

```
FUNCTION calculate_bill(units, fixed_charge=75)\n  VALIDATE units >= 0 AND fixed_charge >= 0\n  energy = 0\n  FOR each slab (limit, rate):\n    
```

7. Python Source Code

Full executable source: programs/electricity_billing.py

```
def calculate_bill(units, fixed_charge=75.0):
    """Progressive tariff: first 100 at 1.50, next 100 at 2.50, rest at 4.00."""
    if units < 0 or fixed_charge < 0:
        raise ValueError("Units and fixed charge must be non-negative.")
    first = min(units, 100) * 1.50
    second = min(max(units - 100, 0), 100) * 2.50
    third = max(units - 200, 0) * 4.00
    energy = first + second + third
    return {"units": units, "energy_charge": round(energy, 2),
            "fixed_charge": round(fixed_charge, 2),
            "total": round(energy + fixed_charge, 2)}

def main():
    print("PROGRESSIVE ELECTRICITY BILLING")
    for units in (0, 100, 150, 250):
        bill = calculate_bill(units)
        print(f"{units} units: energy=■{bill['energy_charge']:.2f}, fixed=■{bill['fixed_charge']:.2f}, total=■{bill['total']:.2f}")

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Zero units	0 units	Energy ■0.00; total ■75.00
2. First slab boundary	100 units	Energy ■150.00; total ■225.00
3. Two slabs	150 units	Energy ■275.00; total ■350.00
4. Above 200 units	250 units	Energy ■600.00; total ■675.00

9. Sample Execution Output

```
PROGRESSIVE ELECTRICITY BILLING
0 units: energy=■0.00, fixed=■75.00, total=■75.00
100 units: energy=■150.00, fixed=■75.00, total=■225.00
150 units: energy=■275.00, fixed=■75.00, total=■350.00
250 units: energy=■600.00, fixed=■75.00, total=■675.00
```

Python output — Case Study 02

```
> python electricity_billing.py
PROGRESSIVE ELECTRICITY BILLING
0 units: energy=₹0.00, fixed=₹75.00, total=₹75.00
100 units: energy=₹150.00, fixed=₹75.00, total=₹225.00
150 units: energy=₹275.00, fixed=₹75.00, total=₹350.00
250 units: energy=₹600.00, fixed=₹75.00, total=₹675.00
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 03: Employee Payroll and Deduction Analysis

1. Problem Statement

Compute an employee's gross salary, provident-fund deduction, income-tax deduction, and net salary from basic pay and allowances.

2. Objectives

• Calculate gross pay from multiple components
• Apply percentage deductions
• Produce a transparent salary breakdown
• Prevent invalid negative salary inputs

3. Assumptions

• HRA is 20% of basic pay; other allowance is 10%.
• PF is 12% of basic pay.
• Illustrative income tax is 5% of gross pay when gross exceeds ■50,000; otherwise zero.
• These are assignment rules, not tax or payroll advice.

4. Algorithm

1. Validate basic pay.
2. Calculate HRA and other allowance.
3. Compute gross salary.
4. Calculate PF and conditional tax.
5. Subtract deductions from gross pay.
6. Return a rounded payroll statement.

5. Flowchart (logical sequence)

Start → Read basic pay → Basic pay non-negative? → Calculate allowances and gross → Calculate PF and conditional tax → Compute net salary → Display payroll → End

6. Pseudocode

```
FUNCTION payroll(basic)\n  VALIDATE basic >= 0\n  hra = basic * 0.20; allowance = basic * 0.10\n  gross = basic + hra + allowance\n  pf = basic *
```

7. Python Source Code

Full executable source: programs/employee_payroll.py

```
def calculate_payroll(basic_pay):
    """Return gross pay, deductions and net pay using the assignment's sample rules."""
    if basic_pay < 0:
        raise ValueError("Basic pay cannot be negative.")
    hra = basic_pay * 0.20
    other_allowance = basic_pay * 0.10
    gross = basic_pay + hra + other_allowance
    pf = basic_pay * 0.12
    tax = gross * 0.05 if gross > 50000 else 0.0
    net = gross - pf - tax
    return {k: round(v, 2) for k, v in {
        "basic_pay": basic_pay, "hra": hra, "other_allowance": other_allowance,
        "gross": gross, "pf": pf, "tax": tax, "net": net}.items()}

def main():
    print("EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS")
    for basic in (20000, 40000, 60000):
        p = calculate_payroll(basic)
        print(f"Basic ■{basic:.2f}: gross=■{p['gross']:.2f}, PF=■{p['pf']:.2f}, tax=■{p['tax']:.2f}, net=■{p['net']:.2f}")

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Below tax threshold	Basic ■20,000	Gross ■26,000; PF ■2,400; tax ■0; net ■23,600
2. Boundary gross ■50,000	Basic ■38,461.53 (gross just below ■50,000)	Tax is zero at gross <= ■50,000
3. Tax applies	Basic ■60,000	Gross ■78,000; PF ■7,200; tax ■3,900; net ■66,900
4. Zero basic	Basic ■0	All amounts ■0.00

9. Sample Execution Output

```
EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS
Basic ■20000.00: gross=■26000.00, PF=■2400.00, tax=■0.00, net=■23600.00
Basic ■40000.00: gross=■52000.00, PF=■4800.00, tax=■2600.00, net=■44600.00
Basic ■60000.00: gross=■78000.00, PF=■7200.00, tax=■3900.00, net=■66900.00
```

Python output — Case Study 03

```
> python employee_payroll.py
EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS
Basic ₹20000.00: gross=₹26000.00, PF=₹2400.00, tax=₹0.00, net=₹23600.00
Basic ₹40000.00: gross=₹52000.00, PF=₹4800.00, tax=₹2600.00, net=₹44600.00
Basic ₹60000.00: gross=₹78000.00, PF=₹7200.00, tax=₹3900.00, net=₹66900.00
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 04: Retail Billing with Multiple Discount Rules

1. Problem Statement

Create a retail checkout calculator that applies a customer discount and an additional bulk discount, then computes tax on the discounted subtotal.

2. Objectives

• Process a cart with multiple items
• Apply ordered discount rules
• Calculate tax after discounts
• Return an itemized receipt

3. Assumptions

• Each item is a (name, unit_price, quantity) tuple.
• A 10% loyalty discount applies when subtotal is at least ■2,000.
• A 5% bulk discount applies when total quantity is at least 10.
• Discounts are sequential percentages on the original subtotal; tax is 5% after discounts.

4. Algorithm

1. Validate each item and calculate subtotal and quantity.
2. Apply loyalty discount if threshold is met.
3. Apply bulk discount if quantity threshold is met.
4. Calculate 5% tax on the discounted amount.
5. Return receipt totals.

5. Flowchart (logical sequence)

Start → Read cart items → Validate prices and quantities → Compute subtotal and quantity → Apply eligible discounts → Compute tax and final total → Print receipt → End

6. Pseudocode

```
FUNCTION checkout(items)\n  VALIDATE prices >= 0 and quantities >= 0\n  subtotal = SUM(price * quantity)\n  loyalty = 10% subtotal IF subtotal >=
```

7. Python Source Code

Full executable source: programs/retail_billing.py

```
def checkout(items):
    """items is an iterable of (name, unit_price, quantity)."""
    subtotal = 0.0
    total_quantity = 0
    for name, price, quantity in items:
        if price < 0 or quantity < 0:
            raise ValueError("Price and quantity cannot be negative.")
        subtotal += price * quantity
        total_quantity += quantity
    loyalty = subtotal * 0.10 if subtotal >= 2000 else 0.0
    bulk = subtotal * 0.05 if total_quantity >= 10 else 0.0
    discounted = subtotal - loyalty - bulk
    tax = discounted * 0.05
    return {k: round(v, 2) for k, v in {
        "subtotal": subtotal, "loyalty_discount": loyalty, "bulk_discount": bulk,
        "discounted_subtotal": discounted, "tax": tax, "total": discounted + tax}.items()}

def main():
    carts = [
        ("Small cart", [("Pen", 20, 2)]),
        ("Loyalty discount", [("Bag", 1200, 2)]),
        ("Bulk discount", [("Notebook", 100, 10)]),
        ("Both discounts", [("Headset", 250, 10)]),
    ]
    print("RETAIL BILLING WITH MULTIPLE DISCOUNT RULES")
    for label, cart in carts:
        r = checkout(cart)
        print(f"{label}: subtotal=■{r['subtotal']:.2f}, discounts=■{r['loyalty_discount']+r['bulk_discount']:.2f}, tax=■{r['tax']:.2f}, total=■

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. No discount	2 pens at ■20	Subtotal ■40; total ■42.00
2. Loyalty only	2 bags at ■1,200	Subtotal ■2,400; loyalty ■240; total ■2,268.00
3. Bulk only	10 notebooks at ■100	Bulk ■50; total ■997.50
4. Both discounts	10 headsets at ■250	Subtotal ■2,500; discounts ■375; total ■2,231.25

9. Sample Execution Output

```
RETAIL BILLING WITH MULTIPLE DISCOUNT RULES
Small cart: subtotal=■40.00, discounts=■0.00, tax=■2.00, total=■42.00
Loyalty discount: subtotal=■2400.00, discounts=■240.00, tax=■108.00, total=■2268.00
Bulk discount: subtotal=■1000.00, discounts=■50.00, tax=■47.50, total=■997.50
Both discounts: subtotal=■2500.00, discounts=■375.00, tax=■106.25, total=■2231.25
```

Python output — Case Study 04

```
> python retail_billing.py
RETAIL BILLING WITH MULTIPLE DISCOUNT RULES
Small cart: subtotal=₹40.00, discounts=₹0.00, tax=₹2.00, total=₹42.00
Loyalty discount: subtotal=₹2400.00, discounts=₹240.00, tax=₹108.00, total=₹2268.00
Bulk discount: subtotal=₹1000.00, discounts=₹50.00, tax=₹47.50, total=₹997.50
Both discounts: subtotal=₹2500.00, discounts=₹375.00, tax=₹106.25, total=₹2231.25
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 05: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance percentage and determine the minimum consecutive classes a student must attend to reach a required attendance percentage.

2. Objectives

• Calculate attendance percentages
• Solve a threshold inequality
• Handle already-eligible and impossible cases
• Explain the result clearly

3. Assumptions

• Attendance policy threshold is 75%.
• Attended classes cannot exceed conducted classes.
• Future classes are assumed to be attended without absence.
• If no classes have been conducted, current percentage is reported as 0%.

4. Algorithm

1. Validate attended and conducted counts.
2. Calculate current percentage.
3. If percentage already meets the threshold, required classes are zero.
4. Otherwise solve (attended+x)/(conducted+x) >= threshold.
5. Round the required number of classes up to the next integer.

5. Flowchart (logical sequence)

Start → Read attended and conducted classes → Validate counts → Calculate current percentage → Already at target? → Solve for minimum future attended classes → Display result → End

6. Pseudocode

```
FUNCTION recovery(attended, conducted, target=0.75)\n  VALIDATE counts and attended <= conducted\n  IF conducted = 0: current = 0 ELSE current =
```

7. Python Source Code

Full executable source: programs/attendance_recovery.py

```
import math

def attendance_recovery(attended, conducted, target=0.75):
    if conducted < 0 or attended < 0 or attended > conducted:
        raise ValueError("Counts must be non-negative and attended <= conducted.")
    if not 0 < target <= 1:
        raise ValueError("Target must be greater than 0 and at most 1.")
    current = attended / conducted if conducted else 0.0
    if current >= target:
        required = 0
    elif target == 1:
        required = math.inf
    else:
        required = math.ceil((target * conducted - attended) / (1 - target))
        required = max(0, required)
    return {"current_percent": round(current * 100, 2),
            "target_percent": round(target * 100, 2), "required_classes": required}

def main():
    print("ATTENDANCE RECOVERY COMPUTATION")
    for attended, conducted in ((18, 30), (20, 25), (0, 0), (35, 50)):
        r = attendance_recovery(attended, conducted)
        print(f"{attended}/{conducted}: current={r['current_percent']:.2f}%, target={r['target_percent']:.0f}%, required future classes={r['requ

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Below target	18 attended / 30 conducted	60%; attend next 18 classes to reach 75%
2. Already eligible	20/25	80%; 0 additional classes
3. No classes yet	0/0	0%; 0 additional classes under current formula (first attended classes can be counted)
4. Exactly target	35/50	70%; attend next 10 classes to reach 75%

9. Sample Execution Output

```
ATTENDANCE RECOVERY COMPUTATION
18/30: current=60.00%, target=75%, required future classes=18
20/25: current=80.00%, target=75%, required future classes=0
0/0: current=0.00%, target=75%, required future classes=0
35/50: current=70.00%, target=75%, required future classes=10
```

Python output — Case Study 05

```
> python attendance_recovery.py
ATTENDANCE RECOVERY COMPUTATION
18/30: current=60.00%, target=75%, required future classes=18
20/25: current=80.00%, target=75%, required future classes=0
0/0: current=0.00%, target=75%, required future classes=0
35/50: current=70.00%, target=75%, required future classes=10
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 06: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of ATM withdrawals and deposits while maintaining balance, rejecting invalid transactions, and preventing overdrafts.

2. Objectives

• Maintain account state
• Validate transaction sequences
• Handle insufficient funds
• Record transaction outcomes

3. Assumptions

• Opening balance cannot be negative.
• Withdrawals and deposits must be positive amounts.
• A withdrawal is rejected if it exceeds the current balance.
• Invalid transactions do not change the balance.

4. Algorithm

1. Validate opening balance.
2. Process transactions in sequence.
3. For each transaction, validate type and amount.
4. Apply deposit or approve/reject withdrawal.
5. Record status and resulting balance.

5. Flowchart (logical sequence)

Start → Read opening balance and transactions → Validate transaction → Deposit? → If withdrawal, sufficient balance? → Update or reject → Record balance after each step → End

6. Pseudocode

```
FUNCTION simulate(balance, transactions)\n  VALIDATE balance >= 0\n  FOR (type, amount) IN transactions:\n    IF amount <= 0: reject\n    ELSE IF
```

7. Python Source Code

Full executable source: programs/atm_simulation.py

```
def simulate_atm(opening_balance, transactions):
    if opening_balance < 0:
        raise ValueError("Opening balance cannot be negative.")
    balance = float(opening_balance)
    ledger = []
    for kind, amount in transactions:
        if amount <= 0:
            status = "REJECTED: amount must be positive"
        elif kind.lower() == "deposit":
            balance += amount
            status = "APPROVED"
        elif kind.lower() == "withdraw":
            if amount <= balance:
                balance -= amount
                status = "APPROVED"
            else:
                status = "REJECTED: insufficient funds"
        else:
            status = "REJECTED: unknown transaction type"
        ledger.append({"type": kind, "amount": amount, "status": status, "balance": round(balance, 2)})
    return {"ledger": ledger, "final_balance": round(balance, 2)}

def main():
    transactions = [("withdraw", 300), ("deposit", 500), ("withdraw", 1500), ("deposit", 0), ("withdraw", 200)]
    result = simulate_atm(1000, transactions)
    print("ATM TRANSACTION SEQUENCE SIMULATION")
    for i, row in enumerate(result["ledger"], 1):
        print(f"{i}. {row['type']} ■{row['amount']:.2f}: {row['status']} | balance ■{row['balance']:.2f}")
    print(f"Final balance: ■{result['final_balance']:.2f}")

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Successful withdrawal	Opening ■1,000; withdraw ■300	Approved; balance ■700
2. Deposit then withdrawal	Opening ■1,000; deposit ■500; withdraw ■1,200	Final balance ■300
3. Insufficient funds	Opening ■1,000; withdraw ■1,500	Rejected; balance remains ■1,000
4. Invalid zero amount	Opening ■1,000; deposit ■0	Rejected; balance remains ■1,000

9. Sample Execution Output

```
ATM TRANSACTION SEQUENCE SIMULATION
1. withdraw ■300.00: APPROVED | balance ■700.00
2. deposit ■500.00: APPROVED | balance ■1200.00
3. withdraw ■1500.00: REJECTED: insufficient funds | balance ■1200.00
4. deposit ■0.00: REJECTED: amount must be positive | balance ■1200.00
5. withdraw ■200.00: APPROVED | balance ■1000.00
```

```
Final balance: ■1000.00
```

Python output — Case Study 06

```
> python atm_simulation.py
ATM TRANSACTION SEQUENCE SIMULATION
1. withdraw ₹300.00: APPROVED | balance ₹700.00
2. deposit ₹500.00: APPROVED | balance ₹1200.00
3. withdraw ₹1500.00: REJECTED: insufficient funds | balance ₹1200.00
4. deposit ₹0.00: REJECTED: amount must be positive | balance ₹1200.00
5. withdraw ₹200.00: APPROVED | balance ₹1000.00
Final balance: ₹1000.00
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 07: Warehouse Inventory Reconciliation

1. Problem Statement

Compare expected stock with physical stock counts and report shortages, overages, and matched items.

2. Objectives

• Represent stock with dictionaries
• Compare expected and counted quantities
• Classify discrepancies
• Calculate inventory value variance

3. Assumptions

• Inventory quantities are non-negative integers.
• Unit cost is non-negative.
• Variance is physical count minus expected stock.
• A negative variance means shortage; positive means overage.

4. Algorithm

1. Validate expected stock and physical count.
2. Iterate over the union of item names.
3. Calculate quantity variance.
4. Classify as MATCHED, SHORTAGE, or OVERAGE.
5. Calculate variance value using unit cost when provided.

5. Flowchart (logical sequence)

Start → Read expected and physical inventory → Validate quantities → Compare each item → Calculate quantity and value variance → Classify mismatch → Display reconciliation report → End

6. Pseudocode

```
FUNCTION reconcile(expected, physical, costs)\n  items = UNION(keys(expected), keys(physical))\n  FOR each item:\n    variance = physical.get(ite
```

7. Python Source Code

Full executable source: programs/inventory_reconciliation.py

```
def reconcile_inventory(expected, physical, unit_costs=None):
    unit_costs = unit_costs or {}
    report = {}
    for item in sorted(set(expected) | set(physical)):
        exp = expected.get(item, 0)
        actual = physical.get(item, 0)
        cost = unit_costs.get(item, 0)
        if not isinstance(exp, int) or not isinstance(actual, int) or exp < 0 or actual < 0:
            raise ValueError("Inventory quantities must be non-negative integers.")
        if cost < 0:
            raise ValueError("Unit costs cannot be negative.")
        variance = actual - exp
        status = "MATCHED" if variance == 0 else ("SHORTAGE" if variance < 0 else "OVERAGE")
        report[item] = {"expected": exp, "physical": actual, "variance": variance,
                        "status": status, "value_variance": round(variance * cost, 2)}
    return report

def main():
    expected = {"Laptop": 10, "Mouse": 40, "Keyboard": 25}
    physical = {"Laptop": 9, "Mouse": 43, "Monitor": 2}
    costs = {"Laptop": 45000, "Mouse": 500, "Keyboard": 900, "Monitor": 8000}
    print("WAREHOUSE INVENTORY RECONCILIATION")
    for item, r in reconcile_inventory(expected, physical, costs).items():
        print(f"{item}: expected={r['expected']}, physical={r['physical']}, variance={r['variance']:+}, {r['status']}, value variance=■{r['value

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Shortage	Expected 10, physical 9, cost ■100	Variance -1; SHORTAGE; value variance -■100
2. Overage	Expected 5, physical 8, cost ■20	Variance +3; OVERAGE; value variance ■60
3. Matched	Expected 4, physical 4	Variance 0; MATCHED
4. Missing expected item	Expected A=3, physical empty	Variance -3; SHORTAGE

9. Sample Execution Output

```
WAREHOUSE INVENTORY RECONCILIATION
Keyboard: expected=25, physical=0, variance=-25, SHORTAGE, value variance=■-22500.00
Laptop: expected=10, physical=9, variance=-1, SHORTAGE, value variance=■-45000.00
Monitor: expected=0, physical=2, variance=+2, OVERAGE, value variance=■16000.00
Mouse: expected=40, physical=43, variance=+3, OVERAGE, value variance=■1500.00
```

Python output — Case Study 07

```
> python inventory_reconciliation.py
WAREHOUSE INVENTORY RECONCILIATION
Keyboard: expected=25, physical=0, variance=-25, SHORTAGE, value variance=₹-22500.00
Laptop: expected=10, physical=9, variance=-1, SHORTAGE, value variance=₹-45000.00
Monitor: expected=0, physical=2, variance=+2, OVERAGE, value variance=₹16000.00
Mouse: expected=40, physical=43, variance=+3, OVERAGE, value variance=₹1500.00
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 08: Student Marks Matrix Analysis

1. Problem Statement

Analyze a matrix of student marks to calculate each student's total and average, subject averages, highest scorer, and pass/fail status.

2. Objectives

• Process nested lists
• Compute row and column statistics
• Identify highest total
• Apply pass thresholds

3. Assumptions

• Each row represents one student and each column a subject.
• Marks are between 0 and 100.
• A student passes only if every subject mark is at least 35.
• The matrix must be rectangular and non-empty.

4. Algorithm

1. Validate matrix dimensions and mark ranges.
2. Calculate each student's total, average, and status.
3. Calculate average marks per subject.
4. Find student with the highest total.
5. Return the analysis.

5. Flowchart (logical sequence)

Start → Read marks matrix → Validate shape and mark range → Calculate row totals and averages → Check pass/fail condition → Calculate subject averages and topper → Display report → End

6. Pseudocode

```
FUNCTION analyze(marks)\n  VALIDATE non-empty rectangular matrix and marks 0..100\n  FOR each student row: total, average, pass if all marks >= 3
```

7. Python Source Code

Full executable source: programs/marks_matrix.py

```
def analyze_marks(marks):
    if not marks or not marks[0]:
        raise ValueError("Marks matrix must be non-empty.")
    cols = len(marks[0])
    if any(len(row) != cols for row in marks):
        raise ValueError("Marks matrix must be rectangular.")
    if any(mark < 0 or mark > 100 for row in marks for mark in row):
        raise ValueError("Marks must be between 0 and 100.")
    students = []
    for index, row in enumerate(marks, 1):
        total = sum(row)
        students.append({"student": index, "total": total, "average": round(total / cols, 2),
                         "status": "PASS" if all(mark >= 35 for mark in row) else "FAIL"})
    subject_averages = [round(sum(row[j] for row in marks) / len(marks), 2) for j in range(cols)]
    topper = max(students, key=lambda s: s["total"])
    return {"students": students, "subject_averages": subject_averages, "topper": topper["student"]}

def main():
    marks = [[80, 75, 90], [45, 60, 55], [90, 92, 88], [70, 30, 80]]
    r = analyze_marks(marks)
    print("STUDENT MARKS MATRIX ANALYSIS")
    for s in r["students"]:
        print(f"Student {s['student']}: total={s['total']}, average={s['average']:.2f}, {s['status']}")
    print("Subject averages:", ", ".join(f"{x:.2f}" for x in r["subject_averages"]))
    print(f"Topper: Student {r['topper']}")

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Student totals	[[80,75,90],[45,60,55]]	Totals 245 and 160; topper Student 1
2. Fail if one subject below 35	[[70,30,80]]	Student 1 FAIL
3. Perfect marks	[[100,100],[100,100]]	Both PASS; averages 100; topper Student 1
4. Subject averages	[[80,60],[40,100]]	Subject averages 60.00 and 80.00

9. Sample Execution Output

```
STUDENT MARKS MATRIX ANALYSIS
Student 1: total=245, average=81.67, PASS
Student 2: total=160, average=53.33, PASS
Student 3: total=270, average=90.00, PASS
Student 4: total=180, average=60.00, FAIL
Subject averages: 71.25, 64.25, 78.25
Topper: Student 3
```

Python output — Case Study 08

```
> python marks_matrix.py
STUDENT MARKS MATRIX ANALYSIS
Student 1: total=245, average=81.67, PASS
Student 2: total=160, average=53.33, PASS
Student 3: total=270, average=90.00, PASS
Student 4: total=180, average=60.00, FAIL
Subject averages: 71.25, 64.25, 78.25
Topper: Student 3
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 09: Password Policy Validation Engine

1. Problem Statement

Validate passwords against a configurable policy including minimum length, uppercase, lowercase, digit, and special-character requirements.

2. Objectives

• Inspect strings character by character
• Use Boolean flags and sets
• Report all failed rules at once
• Avoid displaying or storing actual passwords in logs

3. Assumptions

• Minimum length is 8 characters.
• Password must contain uppercase, lowercase, digit, and at least one special character from the defined set.
• Whitespace is not accepted.
• This educational validator does not replace secure password storage or authentication.

4. Algorithm

1. Check password length.
2. Check for uppercase, lowercase, digit, special character, and whitespace.
3. Collect every failed rule.
4. Return valid status and the failed-rule list.

5. Flowchart (logical sequence)

Start → Read password → Check length and character classes → Collect failed rules → Any failed rules? → Return VALID or INVALID with reasons → End

6. Pseudocode

```
FUNCTION validate(password)\n  errors = []\n  IF length < 8: add length error\n  IF no uppercase: add uppercase error\n  IF no lowercase: add low
```

7. Python Source Code

Full executable source: programs/password_validator.py

```
import string

SPECIALS = set(string.punctuation)

def validate_password(password):
    errors = []
    if len(password) < 8:
        errors.append("minimum length is 8")
    if not any(ch.isupper() for ch in password):
        errors.append("at least one uppercase letter required")
    if not any(ch.islower() for ch in password):
        errors.append("at least one lowercase letter required")
    if not any(ch.isdigit() for ch in password):
        errors.append("at least one digit required")
    if not any(ch in SPECIALS for ch in password):
        errors.append("at least one special character required")
    if any(ch.isspace() for ch in password):
        errors.append("whitespace is not allowed")
    return {"valid": not errors, "errors": errors}

def main():
    samples = [("Strong sample", "Campus@2026"), ("Too short", "Ab1!"),
               ("Missing special", "Campus2026"), ("Contains space", "Campus @2026")]
    print("PASSWORD POLICY VALIDATION ENGINE")
    for label, password in samples:
        result = validate_password(password)
        print(f"{label}: {'VALID' if result['valid'] else 'INVALID'}")
        if result["errors"]:
            print("  Rules to fix: " + "; ".join(result["errors"]))

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Valid password	Campus@2026	VALID; no failed rules
2. Too short	Ab1!	INVALID; length requirement fails
3. Missing special	Campus2026	INVALID; special character required
4. Whitespace present	Campus @2026	INVALID; whitespace not allowed

9. Sample Execution Output

```
PASSWORD POLICY VALIDATION ENGINE
Strong sample: VALID
Too short: INVALID
  Rules to fix: minimum length is 8
Missing special: INVALID
  Rules to fix: at least one special character required
Contains space: INVALID
```

```
Rules to fix: whitespace is not allowed
```

Python output — Case Study 09

```
> python password_validator.py
PASSWORD POLICY VALIDATION ENGINE
Strong sample: VALID
Too short: INVALID
  Rules to fix: minimum length is 8
Missing special: INVALID
  Rules to fix: at least one special character required
Contains space: INVALID
  Rules to fix: whitespace is not allowed
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Case Study 10: Campus Event Financial Planning

1. Problem Statement

Estimate revenue, expenses, profit or loss, and break-even attendance for a campus event based on registrations, ticket price, sponsorship, and fixed and per-attendee costs.

2. Objectives

• Model event revenue and costs
• Calculate profit or loss
• Estimate break-even attendance
• Support event planning decisions

3. Assumptions

• Registration count and costs cannot be negative.
• Revenue = registrations × ticket price + sponsorship.
• Total cost = fixed cost + registrations × variable cost per attendee.
• Break-even is the minimum integer attendance at which ticket revenue plus sponsorship covers costs; if ticket price <= variable cost, it may be impossible without sponsorship.

4. Algorithm

1. Validate all inputs.
2. Calculate registration revenue and total revenue.
3. Calculate fixed and variable costs.
4. Calculate profit or loss.
5. Calculate minimum break-even registrations when possible.
6. Return financial summary.

5. Flowchart (logical sequence)

Start → Read event financial inputs → Validate inputs → Calculate total revenue → Calculate fixed and variable expenses → Compute profit/loss → Determine break-even registrations → Display financial plan → End

6. Pseudocode

```
FUNCTION plan_event(n, ticket, sponsor, fixed, variable)\n  VALIDATE all values >= 0\n  revenue = n*ticket + sponsor\n  cost = fixed + n*variable
```

7. Python Source Code

Full executable source: programs/event_financial_planning.py

```
import math

def plan_event(registrations, ticket_price, sponsorship, fixed_cost, variable_cost):
    values = (registrations, ticket_price, sponsorship, fixed_cost, variable_cost)
    if any(value < 0 for value in values):
        raise ValueError("All financial inputs must be non-negative.")
    ticket_revenue = registrations * ticket_price
    total_revenue = ticket_revenue + sponsorship
    total_cost = fixed_cost + registrations * variable_cost
    profit = total_revenue - total_cost
    if ticket_price > variable_cost:
        break_even = max(0, math.ceil((fixed_cost - sponsorship) / (ticket_price - variable_cost)))
    elif sponsorship >= fixed_cost:
        break_even = 0
    else:
        break_even = None
    return {"ticket_revenue": round(ticket_revenue, 2), "total_revenue": round(total_revenue, 2),
            "total_cost": round(total_cost, 2), "profit": round(profit, 2),
            "status": "PROFIT" if profit > 0 else ("BREAK-EVEN" if profit == 0 else "LOSS"),
            "break_even_registrations": break_even}

def main():
    examples = [
        ("Tech fest", 300, 200, 30000, 40000, 50),
        ("Small event", 50, 100, 0, 7000, 40),
        ("Sponsored event", 100, 100, 10000, 8000, 30),
    ]
    print("CAMPUS EVENT FINANCIAL PLANNING")
    for label, n, ticket, sponsor, fixed, variable in examples:
        r = plan_event(n, ticket, sponsor, fixed, variable)
        be = "Impossible without more sponsorship or a price change" if r["break_even_registrations"] is None else str(r["break_even_registration
        print(f"{label}: revenue=■{r['total_revenue']:.2f}, cost=■{r['total_cost']:.2f}, {r['status']} ■{abs(r['profit']):.2f}, break-even regis

if __name__ == "__main__":
    main()
```

8. Test Cases with Solutions

Test	Input / scenario	Expected result
1. Profitable event	300 registrations; ticket ■200; sponsor ■30,000; fixed ■40,000; variable ■50	revenue ■90,000; cost ■55,000; profit ■35,000; break-even 67
2. Loss event	50 registrations; ticket ■100; sponsor ■0; fixed ■7,000; variable ■40	revenue ■5,000; cost ■9,000; loss ■4,000; break-even 117
3. Sponsorship covers fixed cost	100 registrations; ticket ■100; sponsor ■10,000; fixed ■8,000; variable ■30	revenue ■20,000; cost ■11,000; profit ■9,000; break-even 0
4. Impossible break-even	10 registrations; ticket ■20; sponsor ■0; fixed ■1,000; variable ■25	break-even impossible under current ticket price

9. Sample Execution Output

```
CAMPUS EVENT FINANCIAL PLANNING
Tech fest: revenue=■90000.00, cost=■55000.00, PROFIT ■35000.00, break-even registrations=67
Small event: revenue=■5000.00, cost=■9000.00, LOSS ■4000.00, break-even registrations=117
Sponsored event: revenue=■20000.00, cost=■11000.00, PROFIT ■9000.00, break-even registrations=0
```

Python output — Case Study 10

```
> python event_financial_planning.py
CAMPUS EVENT FINANCIAL PLANNING
Tech fest: revenue=₹90000.00, cost=₹55000.00, PROFIT ₹35000.00, break-even registrations=67
Small event: revenue=₹5000.00, cost=₹9000.00, LOSS ₹4000.00, break-even registrations=117
Sponsored event: revenue=₹20000.00, cost=₹11000.00, PROFIT ₹9000.00, break-even registrations=
0
```

10. Test Execution Result

PASS — all four automated assertions passed.

11. Conclusion

The program implements the documented rules and was checked against the listed test scenarios. Replace sample thresholds with official requirements if supplied by faculty.

Final Submission Checklist

■ Enter your name, roll number, branch and section.
■ Review and understand each program.
■ Run all test modules using python run_all_tests.py.
■ Verify screenshots and replace with local captures if faculty requires them.
■ Upload programs, tests, documentation, screenshots, README and PDF to GitHub.
■ Submit the repository URL to your faculty member / Lab Mentor.

Note: Rules and rates are illustrative academic assumptions, not official policies or professional advice.